

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA Y SISTEMAS DE DATOS

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA
IDENTIFICACIÓN Y OPTIMIZACIÓN DE PATRONES EN 'SWING CHART' EN
MERCADOS FINANCIEROS

Lijun Zhou

2026

Grado en Ingeniería y Sistemas de Datos

TRABAJO DE FIN DE GRADO

Título: Aplicación de técnicas de Machine Learning para la identificación y optimización de patrones en 'Swing Chart' en mercados financieros

Autor: D. Lijun Zhou

Supervisión técnica:

Supervisión académica: Dr. Eduardo López Gonzalo

Departamento: Departamento de Señales, Sistemas y Radiocomunicaciones.

MIEMBROS DEL TRIBUNAL

Presidente:

Vocal:

Secretario:

Suplente:

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:

.....

Fecha de lectura:

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA Y SISTEMAS DE DATOS

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA
IDENTIFICACIÓN Y OPTIMIZACIÓN DE PATRONES EN 'SWING CHART' EN
MERCADOS FINANCIEROS

Lijun Zhou

2026

Resumen

El presente proyecto tiene como objetivo el desarrollo e implementación de un sistema basado en técnicas de Machine Learning orientado al análisis y detección de patrones en “Swing Charts”, una herramienta ampliamente utilizada en el análisis técnico de los mercados financieros. A partir de datos históricos y en tiempo real, el sistema generará representaciones de “Swing Charts” a partir de “Bar Charts” y aplicará algoritmos de aprendizaje supervisado y no supervisado para identificar configuraciones recurrentes, como dobles máximos, dobles mínimos o rupturas de tendencia.

El desarrollo se llevará a cabo principalmente en Python, utilizando librerías como pandas, scikit-learn y matplotlib, además de entornos de conexión con plataformas de trading como Interactive Brokers API (IBKR). El proyecto busca evaluar la capacidad predictiva de los modelos entrenados y su utilidad en la generación de señales de compra o venta dentro de una estrategia de trading algorítmico. Con ello, se pretende aportar un enfoque innovador que combine la solidez del análisis técnico clásico con la adaptabilidad de las técnicas modernas de inteligencia artificial.

Palabras clave

Trading algorítmico, predicción de mercados financieros, automatización, análisis en tiempo real, backtesting.

Summary

XXXXXXXXXX

Keywords

XXXXXXXXXX

Índice

1	Introducción y objetivos	1
1.1	Introducción	1
1.2	Objetivos	1
2	Estado del arte	2
2.1	XXXXXXXXXX	2
3	Desarrollo	3
3.1	Desarrollo del autómata	3
3.1.1	Carga y limpieza	4
3.1.2	Cálculo de características	4
3.1.3	Selección de características	5
3.1.4	Aprendizaje supervisado	5
3.1.5	Aprendizaje no supervisado	5
3.2	Despliegue usando contenedores de Docker	6
3.2.1	Configuración del contenedor principal	6
3.2.2	Bases de datos	6
3.2.3	Alertas y avisos	6

Listado de figuras

Listado de tablas

Todo list

Explicar mejor cómo funciona la función de inferencia	4
Probar a ver otros métodos de votación además de usar la media	5
Decidir el algoritmo a usar para UL	5
Tanto Airflow como MLflow, ver si hay tiempo para implementarlos	6

Glosario

OHLCV - Precios Open High Low y Close y Volumen

XXXX - XXXXXXXXX

Introducción y objetivos

1.1 Introducción

1.2 Objetivos

Estado del arte

2.1 XXXXXXXXXXXX

Desarrollo

3.1 Desarrollo del autómata

El desarrollo del sistema se divide en diferentes fases dentro de la ETL, destacando la carga de datos históricos, su limpieza y filtrado, su transformación en las diferentes características para alimentar a los modelos de aprendizaje automático.

El objetivo es poder generalizar el código, que no importe de qué activos sean los datos el flujo de datos puede reutilizarse.

Para lograr este objetivo he decidido implementar la lógica del código dentro de un paquete instalable de python usando `pip install -e` y he decidido usar la siguiente estructura organizativa.

```

/
├── paths.py
├── get_info
├── data
│   ├── loader.py
│   ├── cleaner.py
│   ├── resampler.py
│   └── to_parquet.py
├── features
├── supervised
├── unsupervised
├── portfolio
├── risk
├── execution
├── logging
└── retrain

```

Hay funciones que están en algunos de los directorios que pueden servir para varias etapas del proceso, principalmente aquellas encargadas de obtener los datos desde los archivos.

A continuación paso a explicar cada una de las fases del flujo de los datos y las funciones utilizadas en ellas.

3.1.1 Carga y limpieza

Se me han facilitado los datos históricos OHLCV de distintos futuros con los que voy a operar.

En un primer análisis me doy cuenta de que los datos temporales no están regularizados al formato UTC, por lo que es el primer cambio que realizo. Usando de referencia las horas de apertura y cierre de cada futuro, agrupo el volumen por horas para cada activo puedo detectar las horas con poca o nula actividad, con ello puedo inferir la zona horaria usada.

Explicar mejor cómo funciona la función de inferencia

Después, decido filtrar los datos que claramente no deberían de estar, estos son los que aparecen en los fines de semana. Los pedidos realoizados en las horas de descanso las mantengo por que pueden haber sido retrasos en la ejecución por lo que son operaciones reales no realizadas en el momento indicado.

Una vez estandarizado el tiempo aprovecho la función de inferencia y añado una priemra columna derivada que indica el día de *trading* aunque al ser una fecha no se usará directamente para el entrenamiento si me ayudará para calcular valores acumulados diarios.

Por otra parte como mi intención es obtener métricas de comparación entre activos decido recortar el número de filas disponibles en los diferentes archivos para tener un conjunto homogéneo. Además para mejorar el rendimiento y disminuir el tamaño de los archivos convierto los datos que estaban en formato csv a parquet.

3.1.2 Cálculo de características

Una vez con los datos OHLCV limpios, llega el momento de aplicar diferentes funciones de diversas librerías para obtener características derivadas que servirán para alimentar a los modelos.

Hay características de diferentes tipos, momento, estructural, temporal, de tendencia, de volatilidad, de volumen y características derivadas de otras. Al ser una estrategia multidía también he hecho uso de los swing charts y de características derivadas de ellas, además de otras características obtenidas de comparar diferentes activos

Para obtener estas características primero agrupo los datos que venían minuto a minuto a agrupaciones de 4 horas, de esto modo se evita tener el ruido de señales tan cortas y es un buen número para una estrategia interdiía donde una posición puede estar abierta varios días

3.1.3 Selección de características

Una vez calculadas todas las características es necesario reducir ese número porque es posible que muchas de ellas estén interrelacionadas

Para ello se aplican varias técnicas, empezando por la más básica, un filtro de alta correlación, además de eliminar las características que puedan predecir el futuro, las no cíclicas o no estacionarias pueden confundir a los modelos indicando que en valor que quizás representaba el día de la semana sea considerado como más importante a mayor número

También filtro usando el algoritmo de LASSO usando un modelo de aprendizaje automático clásico de sklearn que permití mayor explicabilidad

3.1.4 Aprendizaje supervisado

El uso de modelos de aprendizaje supervisado se centra en modelos de clasificación binaria, en este caso en dos etapas, y dentro de cada etapa se utilizan dos modelos de aprendizaje profundo usando PyTorch, LSTM y GRU. Cada uno de los modelos utilizados tiene una función, LSTM ..., por su parte GRU. Para ambas fases se realiza una votación equitativa, si la media supera el umbral indicado para cada fase se pasa a la siguiente

Probar a ver otros métodos de votación además de usar la media

Existe una tercera etapa, en este caso de regresión que sigue la misma lógica, usando los dos modelos

3.1.5 Aprendizaje no supervisado

Pasando ahora a los modelos, primero voy a explicar las dos situaciones donde voy a implementar modelos de aprendizaje no supervisado

En primer lugar para agrupar los diferentes activos en diferentes grupos. En vez de realizar agrupaciones arbitrarias por ejemplo todas las monedas juntas dejo que el sistema decida cuales son los activos más parecidos

Decidir el algoritmo a usar para UL

3.2 Despliegue usando contenedores de Docker

Una vez que se ha desarrollado el autómata y se ha comprobado que su funcionamiento es correcto, he decidido añadir el proyecto a un contenedor de docker para facilitar el despliegue en otros equipos

3.2.1 Configuración del contenedor principal

Por cómo he desarrollado el sistema y por velocidad todo va a estar incluido en un único contenedor.

No requiere demasiados cambios ya que voy a copiar los códigos desarrollados dentro del contenedor

3.2.2 Bases de datos

Al tener el proyecto dentro de un contenedor veo lógico tener una base de datos con diferentes tablas para guardar diferentes métricas

3.2.3 Alertas y avisos

Por último una vez que todo funciona en contenedores he decidido añadir un sistema de visualización de toda la información que se recopila en directo y en diferido

Uso Prometheus, Grafana y Alertmanager este último conectado a otro servicio que se conecta a la API de whatsapp

3.2.4 Orquestación y MLOps

Tanto Airflow como MLflow, ver si hay tiempo para implementarlos

Una vez desplegado un sistema robusto que cuenta con un autómata funcional y con alertas tanto de seguridad como para mostrar información, decido implementar el uso de Airflow como orquestador.

En general el uso de Airflow está muy ligado al uso de MLflow ya que será el uso principal de esta herramienta por mi parte.

En MLflow

Resultados

4.1 XXXXXXXXXXXX

Conclusiones y líneas futuras

5.1 Conclusiones

5.2 Líneas futuras

Por limitaciones del tiempo dejo pendiente dos mejoras

Por una parte añadir una interfaz gráfica independiente de grafana para tener personalización total

Y por otra parte añadir en esa misma página un chatbot usando RASA o un LLM para responder preguntas a partir de toda la información disponible

Aspectos éticos, económicos, sociales y ambientales

6.1 Introducción

6.2 Descripción de impactos relevantes relacionados con el proyecto

6.3 Análisis detallado de alguno de los principales impactos

6.4 Conclusiones

Presupuesto económico

COSTE DE MANO DE OBRA (coste directo)		Horas	Precio/hora	Total
		360	15 €	5.400 €
COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total
Ordenador personal (Software incluido)	1.500,00 €	6	5	150,00 €
Impresora láser	500,00 €	6	5	50,00 €
Otro equipamiento		X	X	X €
COSTE TOTAL DE RECURSOS MATERIALES				200,00 €
GASTOS GENERALES (costes indirectos)	15 %	sobre CD		840,00 €
BENEFICIO INDUSTRIAL	6 %	sobre CD+CI		336.00 €
MATERIAL FUNGIBLE				
Impresión				100,00 €
Encuadernación				300,00 €
SUBTOTAL PRESUPUESTO				7.176,00 €
IVA APLICABLE			21 %	1.507,00 €
TOTAL PRESUPUESTO				8.683,00 €

Bibliografía

Anexos

A XXXXXXXXXXXXXXXXXXXX